

Design and Implementation of a Database Management System for Event Registration and Venue Scheduling System

Problem & approach

Problem

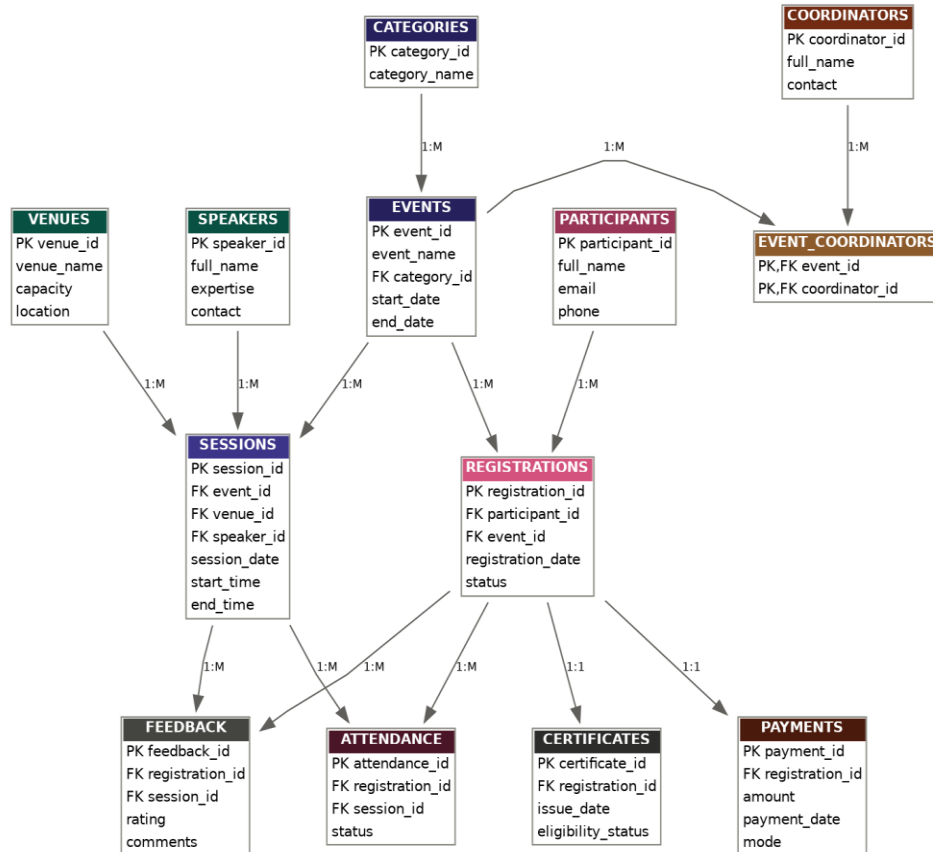
Manual event management leads to venue double-booking, duplicate registrations, unreliable attendance, and certificates issued without checking eligibility.

Approach

A normalized relational database (3NF), with every business rule enforced as a real constraint — built, populated, and queried end to end.

Entity-relationship diagram

13 tables covering the full event lifecycle



What is it showing?

Each box is a table. Every arrow shows relationships meaning "one [box A] can have one or many [box B]."

Registrations sits at the centre — created when a participant signs up for an event. Payment, attendance, certificate and feedback all hang off that one registration.

Events, in turn, are built from categories, sessions (each with one venue + one speaker), and coordinators — linked through the event_coordinators table, since one coordinator can manage several events and one event can have several coordinators.

Key relationships

1

Category (1) → Events (M)

one category (e.g. Workshop) classifies many events

2

Event (1) → Sessions (M)

an event is made up of multiple sessions across its days

3

Venue (1) → Sessions (M) | Speaker (1) → Sessions (M)

each session happens in exactly one venue and has exactly one speaker

4

Event (1) ↔ Coordinator (M:N)

resolved via event_coordinators — one coordinator can manage several events, and vice versa

5

Participant (1) → Registrations (M) | Event (1) → Registrations (M)

a registration links one participant to one event

6

Registration (1) → Payment (0 or 1)

free events never get a payment row

7

Registration (1) → Attendance (M) | Session (1) → Attendance (M)

one attendance row per session a registration is checked against

8

Registration (1) → Certificate (0 or 1)

issued only if the attendance threshold is met

Normalization to 3NF

1NF

Every column holds one atomic value — no repeating groups or lists.

2NF

Every non-key column depends on the whole primary key. All our PKs are single-column IDs, so this is automatic.

3NF

No non-key column depends on another non-key column. A session stores `venue_id`, not the venue's name/capacity — so a capacity change doesn't need updating every session.

Many-to-many relationships get a junction table

Events and Coordinators relate many-to-many, so `event_coordinators(event_id, coordinator_id)` sits between them — a composite-key table with no attributes of its own, purely to resolve the relationship.

Business rules applied → database constraints

No duplicate registration

```
UNIQUE (participant_id, event_id)
```

Non-negative payments

```
CHECK (amount >= 0)
```

Valid attendance status

```
CHECK (status IN ('present','absent'))
```

Feedback rating range

```
CHECK (rating BETWEEN 1 AND 5)
```

Session end after start

```
CHECK (end_time > start_time)
```

Populating the database

Realistic sample data, inserted with plain INSERT statements

```
INSERT INTO events
(event_name, category_id, start_date, end_date)
VALUES
('TechFest 2026', 4, '2026-09-10', '2026-09-12'),
('AI & Cloud Workshop', 2, '2026-09-15', '2026-09-15');

INSERT INTO registrations
(participant_id, event_id, registration_date, status)
VALUES
(1, 1, '2026-08-25', 'confirmed'),
(8, 1, '2026-08-30', 'cancelled'); -- edge case
```

3	5	8	9
Events	Sessions	Participants	Registrations
5	14	8	7
Payments	Attendance	Certificates	Feedback

Every table has data, including deliberate edge cases: a free event with zero payments, and a cancelled registration — so constraints and queries get tested against real awkward cases, not just the happy path.

Query output examples

Actual results from running our queries against the populated database

MySQL Workbench — event_registration_db

```
SELECT e.event_name,
       COUNT(pay.payment_id) AS paid_registrations,
       SUM(pay.amount) AS total_revenue
FROM payments pay
JOIN registrations r ON r.registration_id = pay.registration_id
JOIN events e ON e.event_id = r.event_id
GROUP BY e.event_id, e.event_name
ORDER BY total_revenue DESC;
```

Result Grid

	event_name	paid_registrations	total_revenue
1	AI & Cloud Workshop	3	1500.00
2	Cultural Night	2	200.00

● 2 row(s) returned 0.016 sec

Revenue per event

MySQL Workbench — event_registration_db

```
SELECT p.full_name, e.event_name,
       COUNT(a.attendance_id) AS sessions_recorded,
       SUM(CASE WHEN a.status='present' THEN 1 ELSE 0 END) AS sessions_present,
       ROUND(...) AS attendance_pct
FROM registrations r
JOIN participants p ON p.participant_id = r.participant_id
JOIN events e ON e.event_id = r.event_id
JOIN attendance a ON a.registration_id = r.registration_id
GROUP BY r.registration_id, p.full_name, e.event_name;
```

Result Grid

	full_name	event_name	sessions_recorded	sessions_present	attendance_pct
1	Aditya Kumar	TechFest 2026	3	2	66.7
2	Bhavna Roy	TechFest 2026	3	1	33.3
3	Chirag Patel	TechFest 2026	3	2	66.7
4	Divya Menon	AI & Cloud Workshop	1	1	100.0
5	Farhan Ali	Cultural Night	1	1	100.0

● 8 row(s) returned 0.021 sec

Attendance % per participant

SQL queries covered

Joins

Event seats, registration summary, attendance %, revenue, speaker schedule, feedback ratings — 7 queries

Aggregation

COUNT, SUM, AVG, GROUP BY, HAVING — used across most join queries for reporting

Nested subqueries

Participants who attended every session (NOT EXISTS); events earning above-average revenue (subquery in HAVING)

Self-join check

Verifies no two sessions overlap in the same venue — returns 0 rows, proving the rule holds

Views built on top of the queries

vw_event_seats

Seats booked per event

vw_event_revenue

Total revenue per event

vw_attendance_summary

Attendance % per registration

vw_speaker_schedule

Full schedule per speaker

*Why a view? It saves the query once, so the front-end can just say "SELECT * FROM vw_event_revenue" instead of repeating the joins every time.*

Thank You!!